Pontificia Universidad Católica de Chile
Departamento de Ciencia de la Computación
IIC2613 - Inteligencia Artificial

Reflexión

12 de junio de 2026

1º semestre 2026

Rafael Lorca - 23641789

DCCuidad Automatizada

Parte 1

Área 1: Asignación de camas críticas

Definición: En el curso Introducción a la Ingeniería Matemática, el profesor Alejandro Cataldo nos introdujo el problema de la asignación de camas críticas en salud pública. El problema consiste en asignar eficientemente recursos limitados: medicamentos, personal de la salud, camas, etc a pacientes que requieren atención durante un período determinado. Automatizarlo es relevante porque permite contar con un algoritmo replicable, eficiente y adaptable a distintos contextos hospitalarios, donde las decisiones pueden ser la diferencia entre la vida y la muerte.

Representación: Para nuestra base de conocimiento, considero necesarios las siguientes constantes y predicados (esta será una simplificación del problema real):

Constantes

- médico: Las constantes serían medico1, medico2, medico3, ... Representa a una persona con la profesión de médico, atiende a los pacientes y entrega diagnósticos.
- enfermero: enfermero1, enfermero2, enfermero3, ... Representa a una persona con la profesión de enfermero, atiende a los pacientes suministrando medicamentos y cuidados específicos.
- paciente: paciente1, paciente2, ... Representa a una persona enferma que requiere un diagnóstico y atención médica. Del médico recibe el diagnóstico y de la enfermera cuidados inmediatos. Ocupa una cama crítica por un periodo de tiempo T.

- cama: cama1, cama2, ... Representa un objeto asignado a un paciente.
- medicamento: medicamento1, medicamento2, ... Representa un objeto asignado a un paciente.
- dia: dia1, dia2, dia3, ... Una constante que representa un dia en la clínica (introduce la dimensión temporal, que es clave).

Predicados

- llega(Y,T): El paciente Y llega al hospital el dia T.
- medico(X): X es médico
- enfermero(E): E es enfermero/a.
- diagnóstico(X,Y): Representa que un médico X le hace un diagnóstico a el paciente Y.
- usa_cama(Y,C,T): Representa que un paciente Y usa una cama C el dia T.
- libera_cama(Y,C,T): Representa que el paciente Y libera la cama C el tiempo T.
- cama_disponible(C,T): Representa que la cama C está disponible para su uso el dia T.
- de_turno(X, T): La persona X (puede ser médico o enfermero/a) está de turno en el hospital el día T.
- es_critico(Y): El paciente Y es crítico.
- necesita_medicamento(Y,M): El paciente Y necesita medicamento M.
- hay_stock(M,T): Hay stock de medicamento M el día T.

Reglas: Los datos del mundo real ingresan al sistema como hechos ASP. Por ejemplo, cuando un paciente llega al hospital el sistema registra llega(paciente3, dia5), cuando se libera una cama registra libera_cama(paciente2, cama1, dia5), y al inicio de cada turno se registran los de_turno(X,T) correspondientes. Estos hechos actúan como los "sensores"del sistema, la información captada del mundo real traducida a afirmaciones lógicas. A partir de estos hechos, el sistema deduce automáticamente mediante las siguientes reglas:

1. Todo paciente que llega recibe diagnóstico si hay médico de turno:

$$diagnostico(X, Y) \ :- \ medico(X),\ llega(Y, T),\ de_turno(X, T).$$

2. A partir del diagnóstico se deduce qué medicamento necesita el paciente:

$$necesita_medicamento(Y, M) \ :- \ diagnostico(X, Y),\ disponible_medicamento(M, T).$$

3. Una cama está disponible si ningún paciente la está usando:

$$cama_disponible(C, T) \ :- \ not\ usa_cama(Y, C, T), dia(T).$$

4. Se asigna una cama a un paciente crítico si hay camas disponibles:

$$usa_cama(Y, C, T) :- es_critico(Y),\ cama_disponible(C, T),\ llega(Y, T).$$

5. Un médico solo hace diagnóstico si está de turno:

$$:- diagnostico(X, Y),\ not\ de_turno(X, T),\ llega(Y, T).$$

6. Un paciente no puede ocupar más de una cama simultáneamente:

$$:- usa_cama(Y, C1, T),\ usa_cama(Y, C2, T), C1! = C2.$$

7. Una cama no puede ser usada por dos pacientes al mismo tiempo:

$$:- usa_cama(Y1, C, T), usa_cama(Y2, C, T),\ Y1! = Y2.$$

8. La cama se libera si el paciente se va.

$$cama_disponible(C, T + 1)\ :- libera_cama(Y, C, T).$$

Entre otras reglas más.

Lo que se optimiza en este problema es el predicado **usa_cama(Y,C,T)** ya que se maximiza o se busca maximizar el número de pacientes atendidos respetando las reglas anteriores.

¿Es combinatorio? Creo que este problema sí es un problema combinatorio ya que se tiene que encontrar una solución óptima para una cantidad gigante de posibilidades. Después de indagar un poco, este problema entra en un área específica de los problemas combinatorios, la optimización combinatorial, que es encontrar la mejor solución, normalmente optimizando algún parámetro, en este caso, encontrar una asignación efectiva de pacientes para camas hospitalarias, respetando los horarios del personal y las capacidades de lso recintos de salud. La alta complejidad y también el dinamismo del problema (nuevos pacientes, diagnósticos que se modifican, problemas en la cadena de suministros), hace que considere que sí es un problema bastante dificil de resolver, además, si es que fuese un problema fácil, ya estaría resuelto y aplicado en todos los hospitales del mundo. La gestión eficiente de los pacientes e insumos médicos no solo significan ganancias o eficiencias monetarias, sino que también puede ser la diferencia entre la vida y la muerte para varias personas, lo que hace que sea un problema aún más delicado y urgente de solucionar.

Otras técnicas/disciplinas requeridas: El razonamiento deductivo permite modelar y verificar que una asignación es válida, pero no garantiza que sea óptima. Para encontrar la mejor solución se requieren técnicas de optimización combinatorial como programación lineal entera o metaheurísticas. Igualmente, considero que ASP se parece mucho a la parte de modelación que vemos en el curso de optimización en la universidad, definir variables y relaciones entre las variables es muy parecido a definir constantes y predicados, solamente que se necesita también una función que maximizar o minimizar sujeto a ciertas restricciones, no basta con saber que existe una solución, sino que hay que encontrar la solución óptima.

Área 2: Asignación de salas de prueba

Definición: Nuevamente en el curso Introducción a la Ingeniería Matemática, la tarea que tuvimso que realizar se trataba de encontrar una asignación óptima de salas de prueba. En el contexto universitario es necesario asignar en un número de salas a alumnos para que rindan una prueba. Sería relevante automatizarlo ya que un sistema efectivo podría mejorar el rendimiento de los alumnos, en donde aunque pueda ser casi imposible, se pueden minimizar los ropes de exámenes o interrogaciones, además de una mejor distribución de las salas.

Representación: Para nuestra base de conocimiento, considero necesarios las siguientes constantes y predicados:

Constantes

- ramo: Las constantes serían ramo1, ramo2, ramo3, ... Representa un ramo que se dicta en la universidad.
- sala: sala1, sala2, sala3 ... Representa una sala en la universidad.
- dia: dia1, dia2, dia3, ... Una constante que representa un dia.
- modulo: m1, m2, m3 ... representa los módulos (horarios) para dar la prueba.

Predicados

- capacidad(X, nro) Representa que la sala X tiene capacidad nro, donde nro es un entero. Ejemplo capacidad(sala1, 20) dice que la sala 1 tiene una capacidad de 20 alumnos.
- num_alumnos(R, nro): Representa que el ramo R lo dan nro alumnos. Por ejemplo num_alumnos(IIC2614, 60) dice que el ramo de inteligencia artificial lo dan 60 alumnos.
- ocupada(S,R,M,T): Nos dice que la sala S está siendo ocupada para dar un examen del ramo R en el módulo M el día T.
- libre(S,M,T): Nos dice que la sala S está libre para dar un en el módulo M el día T.
- en_examen(S,R,M,T): Nos dice que se está dando el examen del ramo R en la sala S el módulo M el día T.
- termina_examen(S,R,M,T): Nos dice que el examen del ramo R que se estaba dando en la sala S, el módulo M y el día T ha terminado.

Reglas: Los datos del mundo real ingresan al sistema como hechos ASP. Por ejemplo, cuando se está planificando se registra la información de las salas y los ramos, el sistema registraría capacidad(N8, 50) o num_alumnos(MAT1610, 900), cuando se asigna una sala de clases el sistema registra ocupada(A7, IIC1103, M3, martes). Nuevamente los hechos actúan como los sensores del sistema, dependiendo de lo que ocurre el sistema actúa, sin embargo está limitado por las siguientes reglas:

1. Una sala está disponible el módulo M si ningún ramo la está usando para un examen:

$$libre(S, M, T) \ :- \ not\ en_examen(S, R, M, T),\ modulo(M)\ dia(T).$$

2. Una sala no puede ser usada por dos ramos al mismo tiempo:

$$:- \ ocupada(S, R1, M, T), ocupada(S, R2, M, T),\ R1! = R2.$$

3. La sala se libera si el examen termina.

$$libre(S, M + 1, T) \ :- \ termina_examen(S, R, M, T).$$

4. Un examen del mismo ramo no se puede dar en dias distintos:

$$:- \ en_examen(S, R, M, T1),\ en_examen(S, R, M, T2), T1! = T2$$

Uso de IA: Hay una regla muy importante que me falta y que no cachaba si se podía hacer que es que si se pueden y de hecho, se debe asignar en el mismo módulo y día a un ramo para que de su examen, respetando las capacidades de las salas, sin embargo no hemos visto maneras de "sumar" las restricciones o ocupar desigualdades, sin embargo cuando le pregunté a claude me dijo que la regla se vería de la forma:

$$:- ocupada(S, R, M, T), num_alumnos(R, N),$$

$$\#sum\{Cap, S : ocupada(S, R, M, T), capacidad(S, Cap)\} < N.$$

La regla nos dice que no puede ocurrir que las salas asignadas a un ramo tengan capacidad total menor al número de alumnos de ese ramo.

Lo que se optimiza en este problema es el predicado **ocupada(S,R,M,T)** ya que se busca maximizar el número de pruebas que se dan en un dia, o sea, asignar el mayor número de salas por módulo.

¿Es combinatorio? Con S salas, R ramos y M módulos disponibles, el número de asignaciones posibles crece exponencialmente ya que para cada ramo hay que elegir qué subconjunto de salas usará, en qué módulo y en qué día, respetando que las capacidades sumen suficiente y que no haya conflictos. Este es un problema clásico de bin packing combinado con scheduling, ambos NP-complejos en el caso general. En la práctica universitaria, donde pueden existir decenas de ramos, cientos de salas y múltiples módulos simultáneos, encontrar la asignación óptima manualmente es inviable y por eso creo que sería relevante utilizarlo. En la tarea se sugería intentar mejorar la distribución de exámenes imponiendo más restricciones, como que cursos afines tuviesen separación de días (hubiese sido muy útil en mi caso, ya que tengo 2 días distintos con 2 exámenes esos días).

Otras técnicas/disciplinas requeridas: Al igual que en la asignación de camas, ASP permite verificar que una asignación es válida, pero encontrar la óptima requiere técnicas adicionales. En este caso podemos necesitar nuevamente conocimientos de optimización combinatorial para maximizar la ocupación o para minimizar salas o módulos usados. Por otra parte creo que es necesario tener algún conocimiento en gestión o bases de datos, ya que se necesita recopilar información sobre la capacidad de las salas de un establecimiento educacional o de la cantida de alumnos que tomaron esos ramos en el semestre, etc, por lo que se necesitaría integración del algoritmo con una base de datos.

Referencias

"Wikipedia. (s. f.). Optimización combinatoria. Recuperado el 11 de junio de 2026, de `https://es.wikipedia.org/wiki/Optimizaci%C3%B3n_combinatoria`"